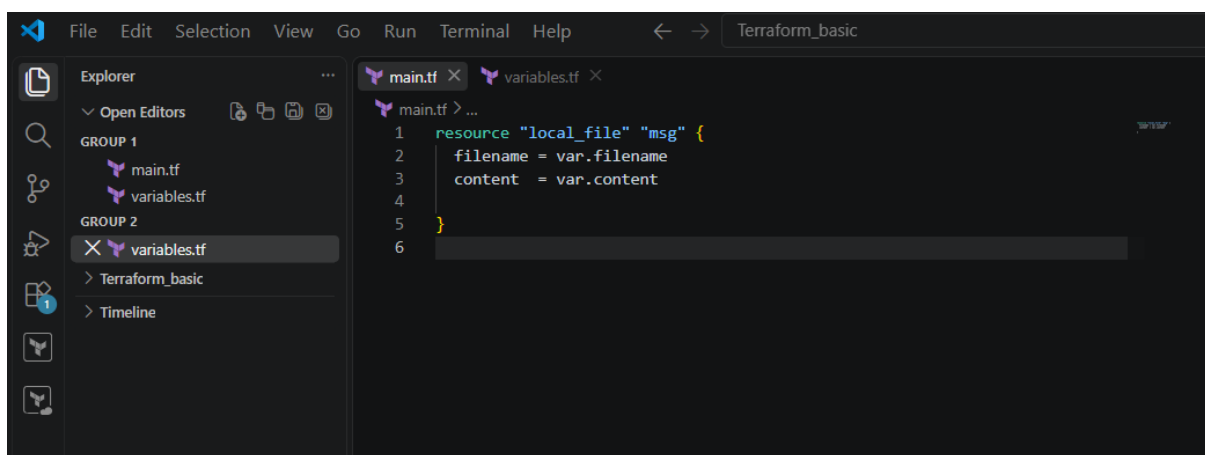


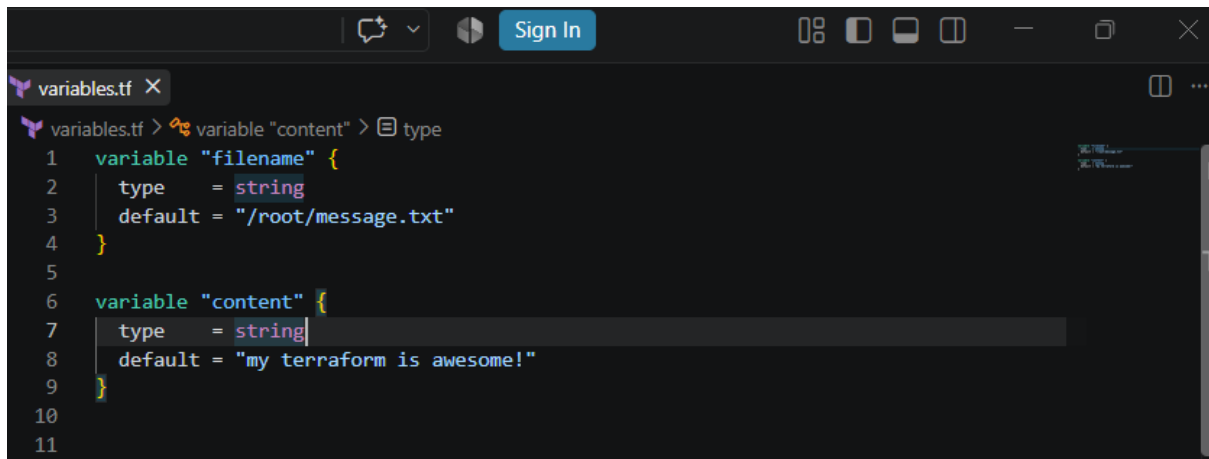
Execute the script shown in the video.

1) By using variables.tf file

- Create a file named **main.tf**
- Add this Terraform code inside main.tf:
- ```
resource "local_file" "msg" {
```
- ```
  filename = var.filename
```
- ```
 content = var.content
```
- ```
}
```
- Save the file.



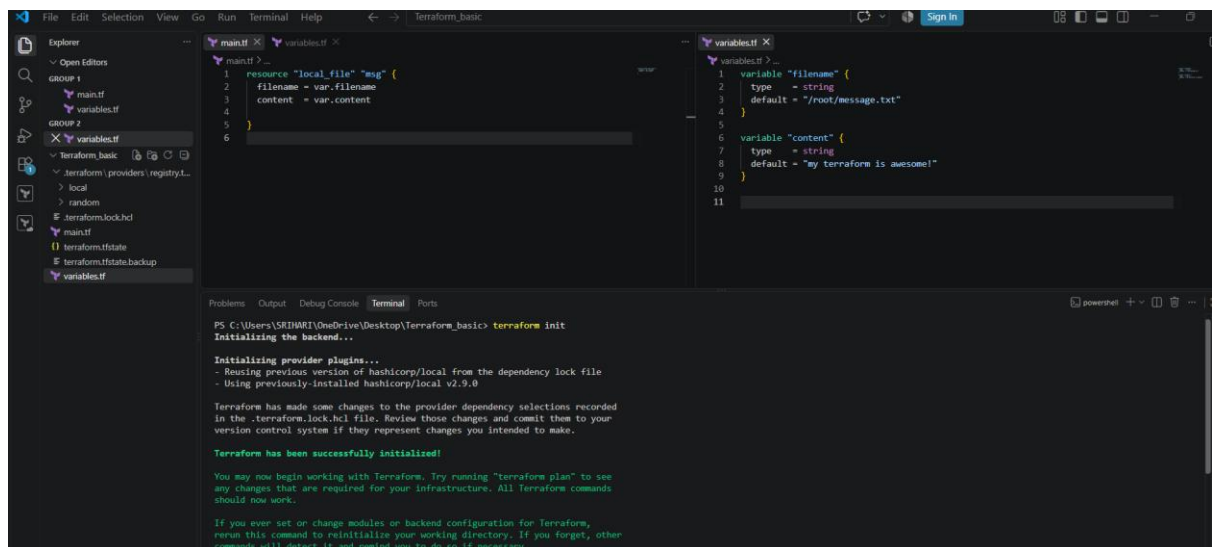
- Create a new file named **variables.tf**
- Add filename variable:
- ```
variable "filename" {
```
- ```
  type = string
```
- ```
 default = "/root/message.txt"
```
- ```
}
```
- Add content variable:
- ```
variable "content" {
```
- ```
  type = string
```
- ```
 default = "Terraform is awesome!"
```
- ```
}
```
- Save the file.



The screenshot shows the VS Code editor with the file `variables.tf` open. The code defines two variables: `filename` and `content`. The `filename` variable is a string with a default value of `"/root/message.txt"`. The `content` variable is also a string with a default value of `"my terraform is awesome!"`. The cursor is positioned at the end of the `content` variable definition.

```
1 variable "filename" {
2   type    = string
3   default = "/root/message.txt"
4 }
5
6 variable "content" {
7   type    = string
8   default = "my terraform is awesome!"
9 }
10
11
```

- Open Terminal in VS Code.
- Go to your Terraform project folder.
- Run:
 1. `terraform init`
- Terraform will download required provider plugins.



The screenshot shows the VS Code editor with the file `main.tf` open. The code defines a resource `local_file` named `msg` with attributes `filename` and `content` both set to `var.filename` and `var.content` respectively. The terminal at the bottom shows the output of the `terraform init` command, indicating that the backend has been initialized and the provider plugins have been downloaded.

```
1 resource "local_file" "msg" {
2   filename = var.filename
3   content  = var.content
4 }
5
6
```

Terminal Output:

```
PS C:\Users\SRINANI\OneDrive\Desktop\Terraform_basics> terraform init
Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/local from the dependency lock file
- Using previously-installed hashicorp/local v2.9.0

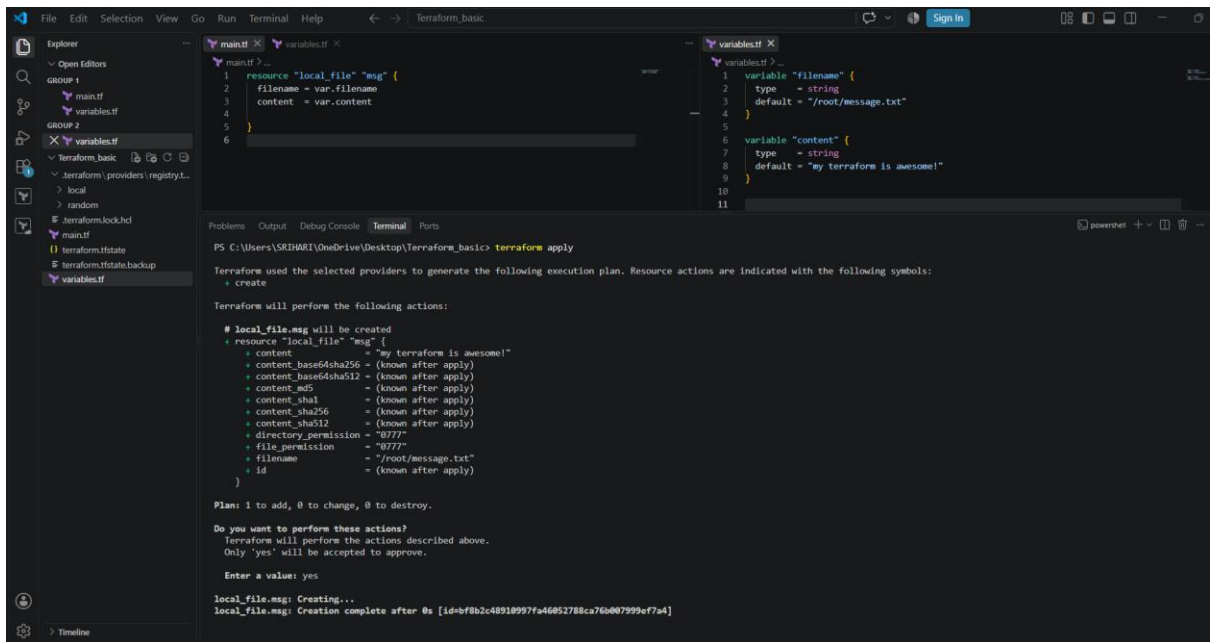
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review these changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

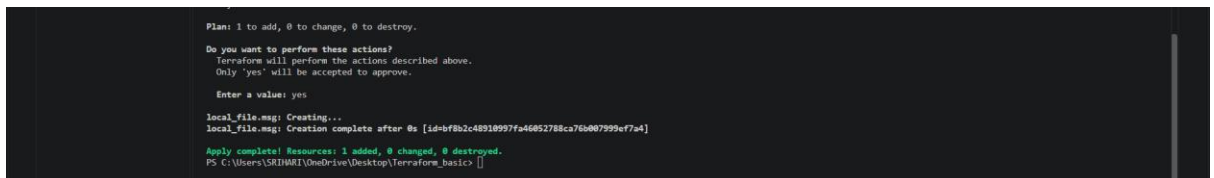
If you ever set or change modules or backend configuration for Terraform,
run this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

- Run the apply command:
 1. `terraform apply`
- Terraform will generate an execution plan.
- You will see output like:
- `local_file.msg` will be created
- filename and content details
- Terraform will ask for confirmation to continue

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project structure with folders 'GROUP 1' and 'GROUP 2', and files 'main.tf', 'variables.tf', 'Terraform_basic', 'Terraform_providers.registry.t...', 'local', 'random', 'Terraform.lock.hcl', 'main.tf', 'Terraform.tfstate', 'Terraform.tfstate.backup', and 'variables.tf'. The main editor area has two tabs: 'main.tf' and 'variables.tf'. The 'main.tf' tab is active, showing a Terraform resource definition for 'local_file' named 'msg'. The 'variables.tf' tab is also visible, showing variable definitions for 'filename' and 'content'. The bottom panel shows the 'Terminal' output, which displays the command 'terraform apply' and its output. The output indicates that Terraform will create a resource 'local_file' named 'msg' with various attributes like 'content', 'content_base64sha256', 'content_base64sha512', 'content_md5', 'content_sha1', 'content_sha256', 'content_sha512', 'directory_permission', 'file_permission', 'filename', and 'id'. It also shows the plan: 1 to add, 0 to change, 0 to destroy. The user is prompted to confirm the actions, and the output shows the resource being created successfully.

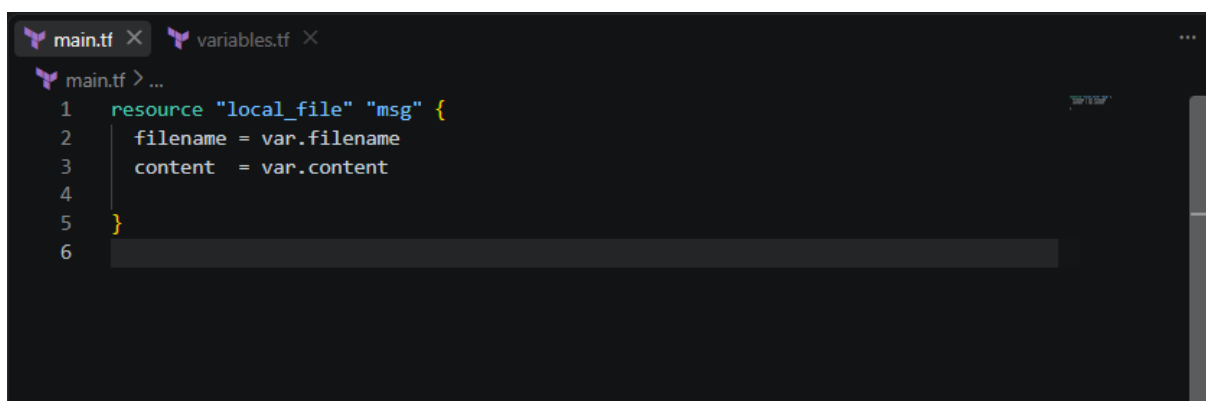
Final output shows:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

A screenshot of the Visual Studio Code terminal window. It shows the output of the 'terraform apply' command. The output includes the plan: 1 to add, 0 to change, 0 to destroy. It prompts the user to confirm the actions, and the user enters 'yes'. The output shows the resource 'local_file' being created successfully. The final line of the output is 'Apply complete! Resources: 1 added, 0 changed, 0 destroyed.' followed by the prompt 'PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basic>'.

2) By using interactive mode.

- Open **main.tf** in VS Code.
- Add the Terraform local_file resource code:
- resource "local_file" "msg" {
- filename = var.filename
- content = var.content
- }
- This means Terraform will take values from variables.tf (or from user input).
- Save

A screenshot of the Visual Studio Code editor showing the 'main.tf' file. The file contains the following Terraform resource definition for 'local_file' named 'msg':

```
1 resource "local_file" "msg" {  
2   filename = var.filename  
3   content  = var.content  
4 }  
5  
6
```

The 'variables.tf' file is also open in the background, showing the variable definitions for 'filename' and 'content'.

- Open **variables.tf**
- Define variable for filename:
- variable "filename" {
- type = string
- }
- Define variable for content:
- variable "content" {
- type = string
- }
- Here no default value is given, so Terraform will ask during apply.
- Save the file.

```

variables.tf
1  variable "filename" {
2      type = string
3  }
4
5
6  variable "content" {
7      type = string
8  }
9
10
11

```

- Open terminal and run:
- terraform apply
- Terraform asks for content, enter value (example):
- Enter a value: hello srihari
- Terraform asks for filename, enter value (example):
- Enter a value: hari.txt
- Terraform refreshes current state and shows plan.
- It shows **local_file.msg must be replaced** because content changed → triggers replacement.

```

PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform apply
var.content
Enter a value: hello srihari

var.filename
Enter a value: hari.txt

local_file.msg: Refreshing state... [id=b782c48918997fa46052788ca76b007999ef7a4]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/. destroy and then create replacement

Terraform will perform the following actions:

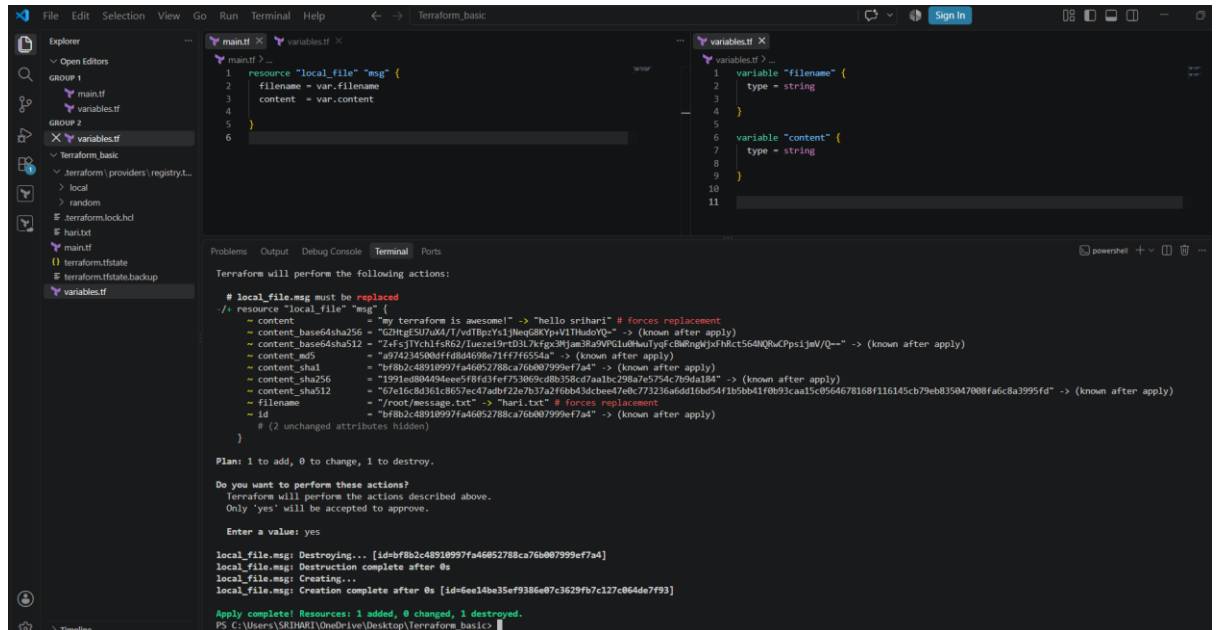
# local_file.msg must be replaced
./ resource "local_file" "msg" {
  content      = "my terraform is awesome!" -> "hello srihari" # forces replacement
  content_base64sha256 = "G0Htg5U0/a44//v018pzy61JNeq8Xp+V1Hud0/Q=" -> (known after apply)
  content_base64sha512 = "74f53jVxh1f802/fuees19e-dt3l7Afp38j3m3Wp9V9j0lshw0jyf-d8hng8djsi8ct5640QwCPpsijw/Q=" -> (known after apply)
  content_md5      = "a974234500dffd84698e71ff7f6554a" -> (known after apply)
  content_sh1      = "bf882c48918997fa46052788ca76b007999ef7a4" -> (known after apply)
  content_sha256    = "1991e898404e0ee5f8f43f64753009cd0b358cd7aallbc298a7e5754c709da184" -> (known after apply)
  content_sha512    = "57e16c84361c8657ec47adbf22e7b3742f6bb43dcbee474b773236a6dd16b54f1b5bb41f0b93ca15c9564678158f116145cb79eb835047008fa6c8a3995fd" -> (known after apply)
  filename         = "/root/message.txt" -> "hari.txt" # forces replacement
  id              = "bf882c48918997fa46052788ca76b007999ef7a4" -> (known after apply)
}
# (2 unchanged attributes hidden)

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?

```

- After plan output Terraform asks:
 1. Do you want to perform these actions?
 2. Enter a value:
- Type:
 1. yes
- Terraform will **destroy the old file resource** and then **create a new one**.
- Output shows:
destroying → Creating → Apply complete



```

Terraform will perform the following actions:

# local_file.msg must be replaced
./ resource "local_file" "msg" {
  ~ content      = "hey terraform is awesome!" -> "hello srihari" # forces replacement
  ~ content_base64sha256 = "G2HtgESU7uA/1/vd1BpZy51jNeqG8YpV1ThudoQ==" -> (known after apply)
  ~ content_base64sha512 = "ZafsjYvch1F8R6Z/1ueze19rtD3L7kFgx3Mjam3a20VPG1u8kulyqfC8aWngqjxfH8ct5640QhuCPspijw/Q==" -> (known after apply)
  ~ content_md5 = "a7d23459a8ff1d8a499871ff7f65da" -> (known after apply)
  ~ content_sha1 = "bf8b2c48918997fa46852788ca76b007999ef7a4" -> (known after apply)
  ~ content_sha256 = "191ed80449eeef8fd3eff753069cd8b358cd7aabc258a7e5754c7b9da184" -> (known after apply)
  ~ content_sha512 = "67e16c84361d857ec47adb72267b3742f18b043dbee478bc773236a6dd16bd54f1b5bb41f0b93caal5c0564678168f116145cb79eb835047088fa6c8a3995fd" -> (known after apply)
  ~ filename = "/root/message.txt" -> "hurl.txt" # forces replacement
  ~ id = "bf8b2c48918997fa46852788ca76b007999ef7a4" -> (known after apply)
  # (2 unchanged attributes hidden)
}

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

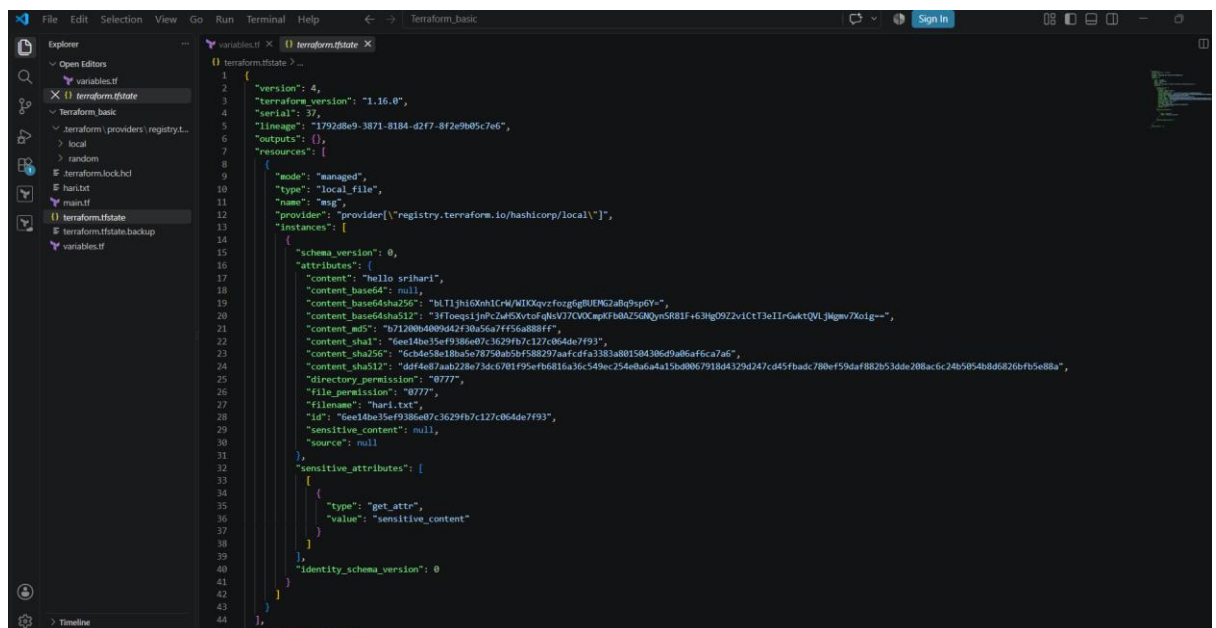
Enter a value: yes

local_file.msg: Destroying... [id=bf8b2c48918997fa46852788ca76b007999ef7a4]
local_file.msg: Destruction complete after 0s
local_file.msg: Creating...
local_file.msg: Creation complete after 0s [id=6ee14be35ef9386e07c3629fb7c127c864de7f93]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\Users\SRIHARI\OneDrive\Desktop\terraform_basics>

```

- Open the file **terraform.tfstate**
- You can see stored attributes for **local_file.msg**.
- The updated content appears inside state file like:
- "content": "hello srihari"
- This proves Terraform stored the latest applied values in state



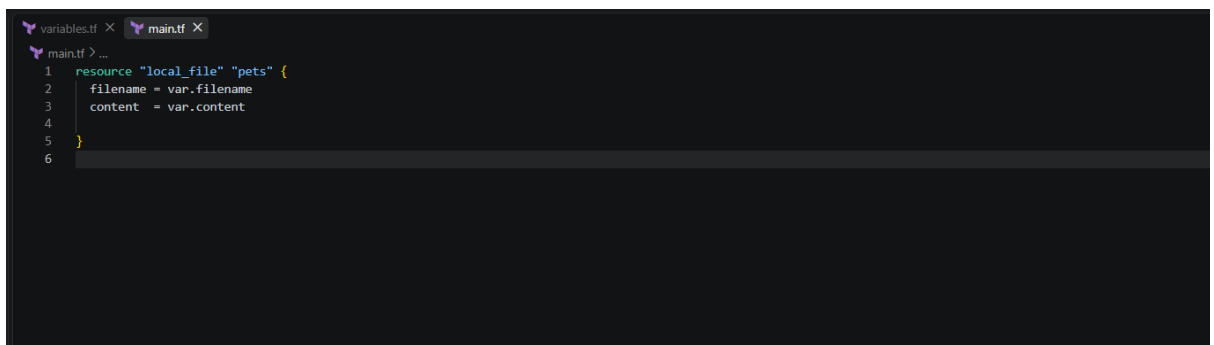
```

{
  "version": 4,
  "terraform_version": "1.16.0",
  "serial": 37,
  "lineage": "1792d8e9-3871-8184-d2f7-8f2e9b05c7e6",
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "local_file",
      "name": "msg",
      "provider": "[\"registry.terraform.io/hashicorp/local\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "content": "hello srihari",
            "content_base64": null,
            "content_base64sha256": "blT1jhi6Xeh1CwH/WIKXqyzfzrgf6gRUEMG2abg9sp6Y+",
            "content_base64sha512": "3fToeqijpKzWd5XvtofqHvJ7CVOCapKfB0A25GQyn5R81F+63Hg92Zv1CtT3eIirGwctQVl_jlgwvXoig==",
            "content_md5": "7b1200b4809d42130a56a7ff56a88ff",
            "content_sha1": "6ee14be35ef9386e07c3629fb7c127c864de7f93",
            "content_sha256": "6cd4e58e18ba5e7d79b5b5f588297aafcdfa3383a801504386d9a8afca7a6",
            "content_sha512": "dd4e87aab228e73dc6701f95efb6816a36c549ec254eb4a4a15b08067918d4329d247cd45fbadc780ef59daf882b53dde208ac6c24054b4bd826bf5e88a",
            "directory_permission": "0777",
            "file_permission": "0777",
            "filename": "hurl.txt",
            "id": "6ee14be35ef9386e07c3629fb7c127c864de7f93",
            "sensitive_content": null,
            "source": null
          },
          "sensitive_attributes": [
            {
              "type": "get_attr",
              "value": "sensitive_content"
            }
          ],
          "identity_schema_version": 0
        }
      ]
    }
  ],
  "check_requirements": null
}

```

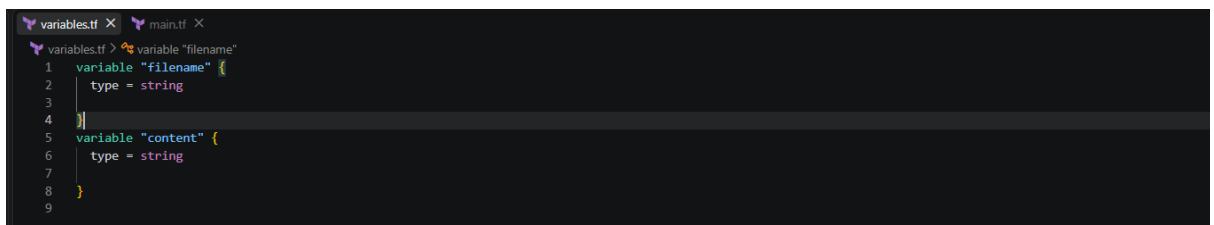
3) Command line flags

- Open **main.tf** in VS Code.
- Create the local file resource with variable values:
- `resource "local_file" "pets" {`
- `filename = var.filename`
- `content = var.content`
- `}`
- Save the file.



```
variables.tf x main.tf x
main.tf > ...
1 resource "local_file" "pets" {
2   filename = var.filename
3   content  = var.content
4
5 }
6
```

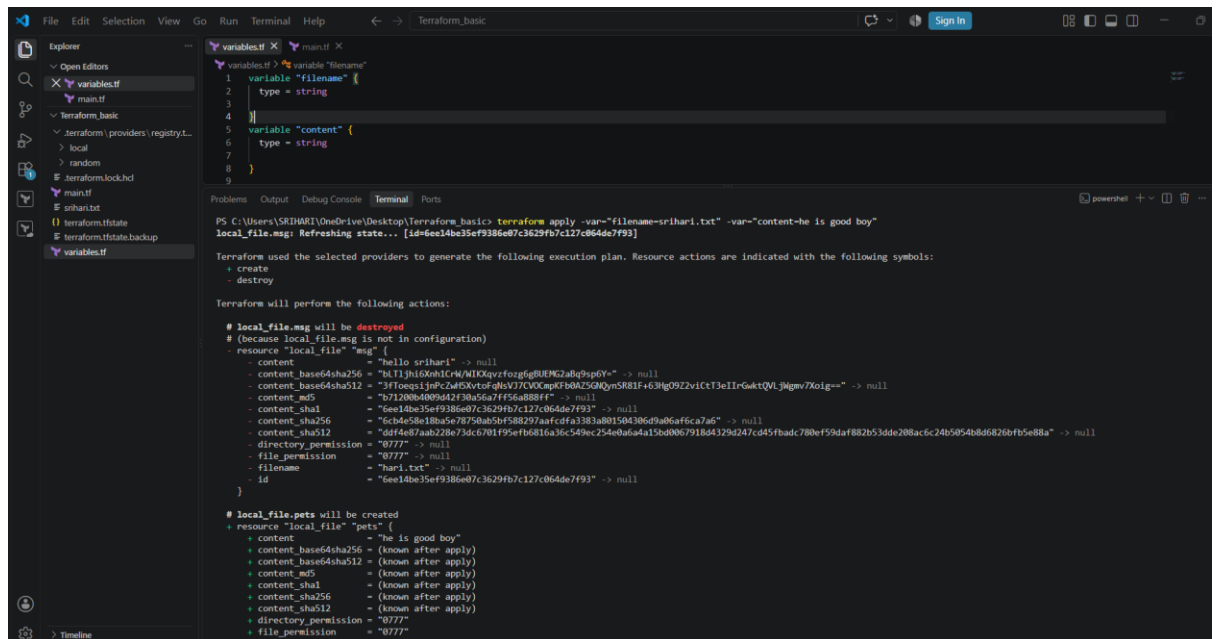
- Open **variables.tf**
- Define filename variable:
 1. `variable "filename" {`
 2. `type = string`
 3. `}`
- Define prefix variable:
 1. `variable "content" {`
 2. `type = string`
 3. `}`
- Save the file



```
variables.tf x main.tf x
variables.tf > variable "filename"
1 variable "filename" {
2   type = string
3 }
4
5 variable "content" {
6   type = string
7 }
8
9
```

- Open terminal inside the same Terraform folder.
- Run Terraform apply by passing values directly:
- `terraform apply -var="filename=srihari.txt" -var="content=he is good boy"`
- Terraform will show the execution plan (`local_file.pets` will be created).
- When it asks confirmation, type:
- `yes`

Terraform creates the resource and shows **Apply complete**.



The screenshot shows the VS Code interface with the Explorer on the left showing the file structure. The main editor displays the `variables.tf` file. The terminal at the bottom shows the output of the `terraform apply` command. The output indicates that the `local_file` resource `"msg"` is being created with the specified content. The output also shows the `local_file.pets` resource being created with the specified content.

```
PS C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform apply -var="filename=srihari.txt" -var="content=he is good boy"
local_file.msg: Refreshing state... [id=6ee14be35ef9386e07c3629fb7c127c064de7f93]

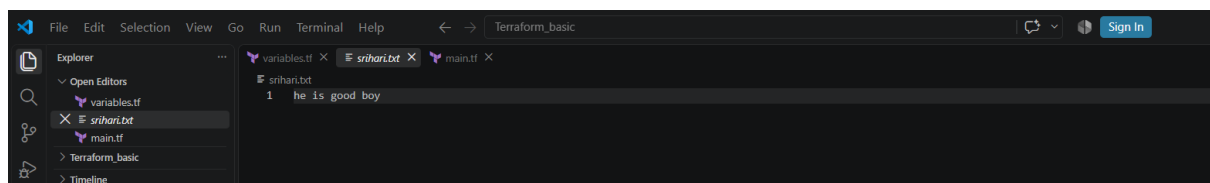
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
- destroy

Terraform will perform the following actions:

# local_file.msg will be destroyed
# (because local_file.msg is not in configuration)
resource "local_file" "msg" {
  content      = "hello srihari" -> null
  content_base64sha256 = "b171jhl60h41c4m4MfKqyrfogp6p8UEMC2dRqhp6Yv" -> null
  content_base64sha512 = "1f70eqs1np72d45kxvaf-dn3v17Cv00npkF80A75GQym5R81F463hg92ZviCtT3eIlrGuktQVJ3gm7Xoig==" -> null
  content_md5      = "b71200b4009d42f30a56a7ff56a888ff" -> null
  content_sh1      = "6ee14be35ef9386e07c3629fb7c127c064de7f93" -> null
  content_sh256    = "6cb4e58e18b5e78750ab58f588227aafcfaf3383a081504386d7b86af6ca7a6" -> null
  content_sh512    = "d4f4e87aab228e73dc6701f95efb816a36c549nc254e06a4a13bd006791864329d247cd45fbad780ef59daf882b53dde208acc24b50954b8d6826bfb5e88a" -> null
  directory_permission = "0777" -> null
  file_permission     = "0777" -> null
  filename            = "srihari.txt" -> null
  id                  = "6ee14be35ef9386e07c3629fb7c127c064de7f93" -> null
}

# local_file.pets will be created
+ resource "local_file" "pets" {
+   content      = "he is good boy"
+   content_base64sha256 = (known after apply)
+   content_base64sha512 = (known after apply)
+   content_md5      = (known after apply)
+   content_sh1      = (known after apply)
+   content_sh256    = (known after apply)
+   content_sh512    = (known after apply)
+   directory_permission = "0777"
+   file_permission     = "0777"
}
```

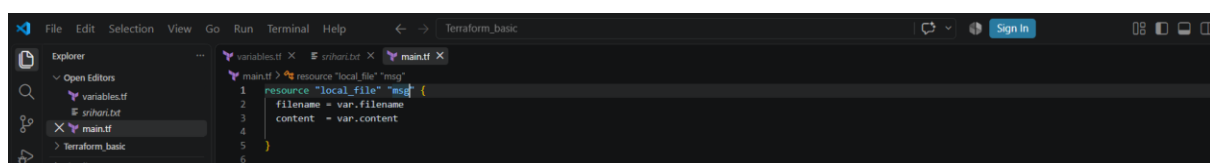
- After apply, you will see a new file created: **srihari.txt**
- Open the file in VS Code.
- It contains the generated content like:
 1. He is good boy
- This confirms Terraform successfully created the file with the passed variable value



The screenshot shows the VS Code interface with the Explorer on the left showing the file structure. The main editor displays the `srihari.txt` file, which contains the text "he is good boy".

4) Environment variables

- Open **main.tf** in VS Code.
- Add Terraform code to create local file using variables:
- resource "local_file" "msg" {
- filename = var.filename
- content = var.content
- }
- Save the file

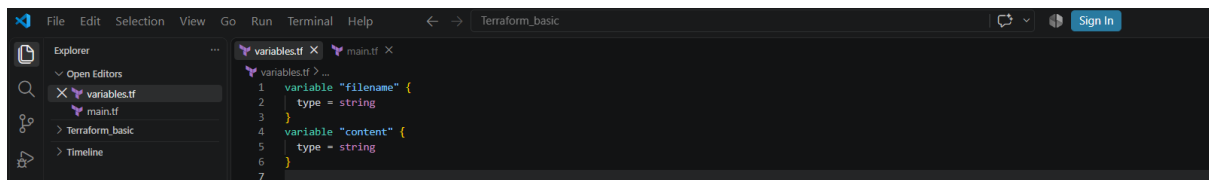


The screenshot shows the VS Code interface with the Explorer on the left showing the file structure. The main editor displays the `main.tf` file, which contains the Terraform code to create a local file using variables.

```
1 resource "local_file" "msg" {
2   filename = var.filename
3   content  = var.content
4 }
5
6
```

- Open **variables.tf**

- Define filename variable:
- `variable "filename" {`
- `type = string`
- `}`
- Define content variable:
- `variable "content" {`
- `type = string`
- `}`
- No default values given, so Terraform will take values from CLI/env/input.
- Save the file.

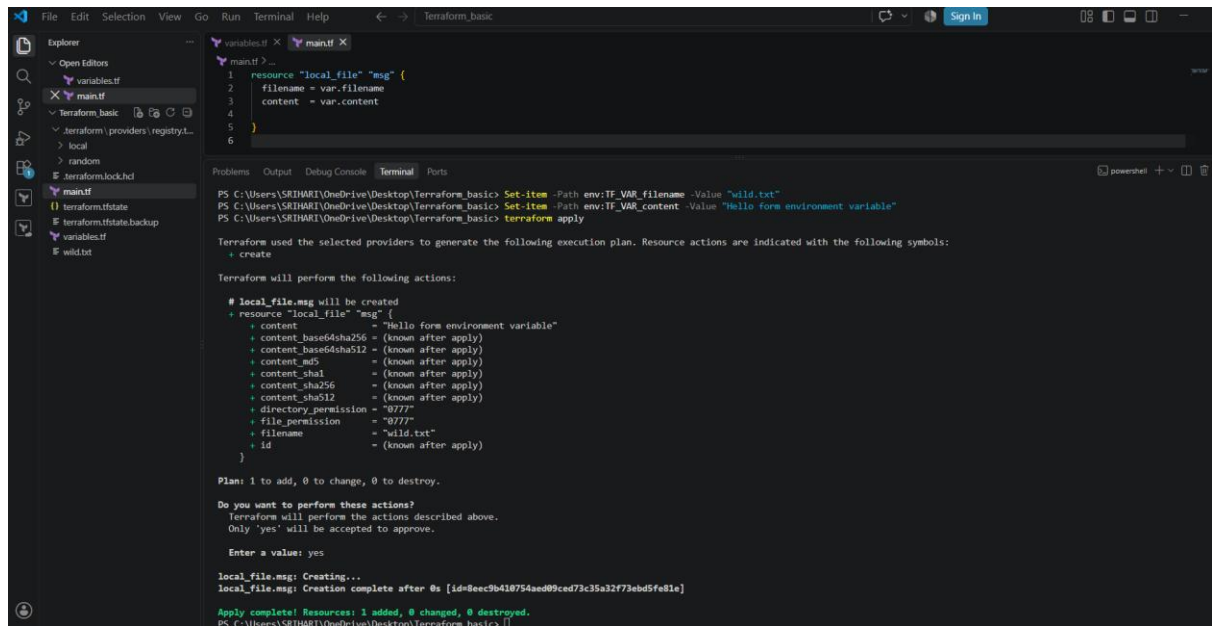


- Open PowerShell terminal in VS Code.
- Set Terraform input variables using environment variables:
- `Set-Item -Path env:TF_VAR_filename -Value "wild.txt"`
- `Set-Item -Path env:TF_VAR_content -Value "Hello from Environment Variable"`
- Run Terraform apply:
- `terraform apply`

Terraform creates execution plan:

1. `local_file.msg` will be created
2. `filename` → `wild.txt`
3. `content` → `Hello from Environment Variable`

Terraform asks confirmation to proceed.



```
File Edit Selection View Go Run Terminal Help
variables.tf main.tf
main.tf
1 resource "local_file" "msg" {
2   filename = var.filename
3   content  = var.content
4 }
5
6

P5 C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> Set-Item -Path env:TF_VAR_filename -Value "wild.txt"
P5 C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> Set-Item -Path env:TF_VAR_content -Value "Hello from environment variable"
P5 C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

+ local_file.msg will be created
+ resource "local_file" "msg" {
  + content          = "Hello from environment variable"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5       = (known after apply)
  + content_shal      = (known after apply)
  + content_sha256    = (known after apply)
  + content_sha512    = (known after apply)
  + directory_permission = "0777"
  + file_permission   = "0777"
  + filename          = "wild.txt"
  + id                = (known after apply)
}

Plan: 1 to add, 0 to change, 0 to destroy.

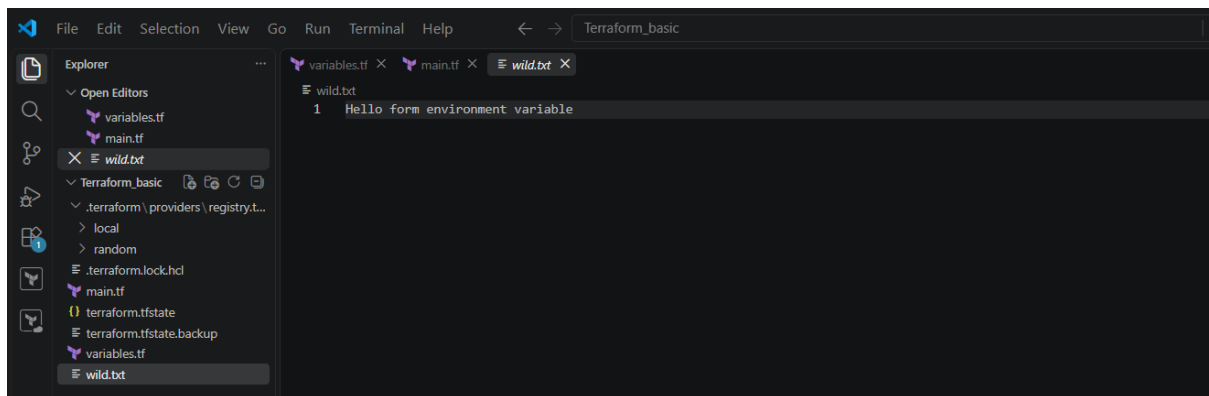
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

local_file.msg: Creating...
local_file.msg: Creation complete after 0s [id=8eec9b418754eed09ced73c35a32f73abd5fe81e]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
P5 C:\Users\SRIHARI\OneDrive\Desktop\Terraform_basics> |
```

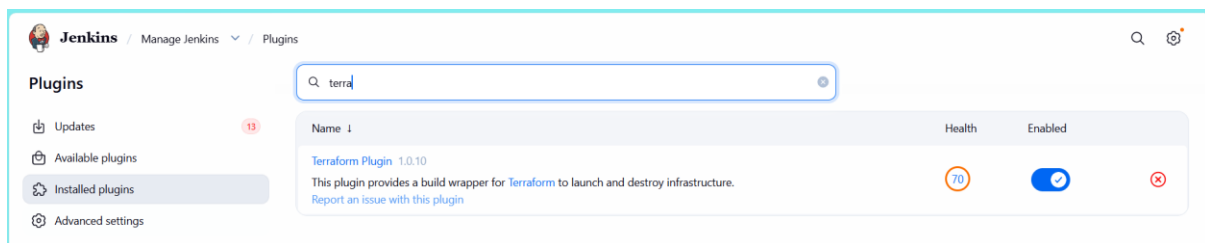
- When it prompts:
- Enter a value:
- type:
- yes
- Terraform creates the file and shows:
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
- Open the generated file **wild.txt**
- Verify output inside file:
- Hello from Environment Variable



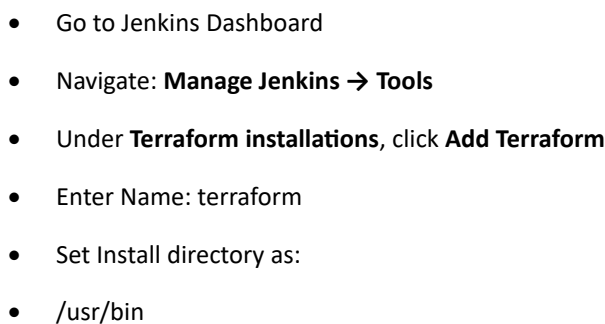
```
File Edit Selection View Go Run Terminal Help
variables.tf main.tf wild.txt
wild.txt
1 Hello from environment variable
```

Integrate **Terraform** in Jenkins using the **Terraform plugin**.

- Login to **Jenkins Dashboard**
- Go to: **Manage Jenkins → Plugins**
- Click **Installed plugins / Available plugins**
- Search for **Terraform**
- Install/Enable **Terraform Plugin**
- Confirm plugin status is **Enabled**



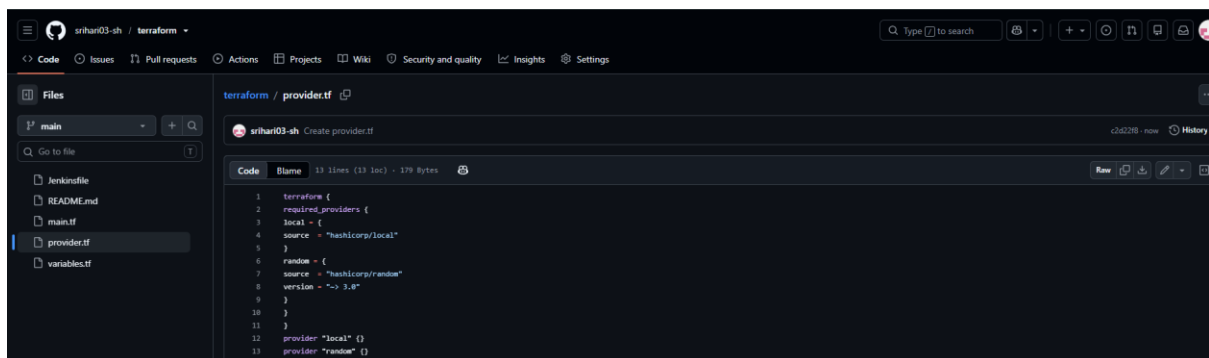
- Login to Jenkins server using SSH (root/user)
- Download Terraform binary (zip) inside /opt or required path
- Unzip the terraform package:
- unzip terraform_<version>_linux_amd64.zip
- Move terraform binary to /usr/bin:
- mv terraform /usr/bin/
- Verify installation:
- terraform version
- Confirm terraform path:
- which terraform
- ls -l /usr/bin/terraform



The screenshot shows two side-by-side windows. The left window is a terminal session on an Amazon Linux 2023 instance. It shows the user running 'terraform -v' which outputs 'Terraform v1.16.0 on linux_amd64'. The user then runs 'terraform update-self' which outputs 'Your version of Terraform is out of date! The latest version is 1.16.1. You can update by downloading from https://developer.hashicorp.com/terraform/install'. The user then runs 'find / -name terraform' which outputs the path '/usr/bin/terraform'. The right window is the 'Manage Jenkins' page, specifically the 'Terraform installations' tab. It shows a table with one entry: 'terraform' installed at '/usr/bin'. The 'Install automatically' checkbox is unchecked. At the bottom, there are 'Save' and 'Apply' buttons.

- Open your GitHub repo (Terraform project)
- Edit/Create file **provider.tf**
- Add required providers block (example):
- terraform {
- required_providers {

- local = {
- source = "hashicorp/local"
- }
- random = {
- source = "hashicorp/random"
- }
- }
- }
- provider "local" {}
- provider "random" {}
- Commit and push the changes to repository



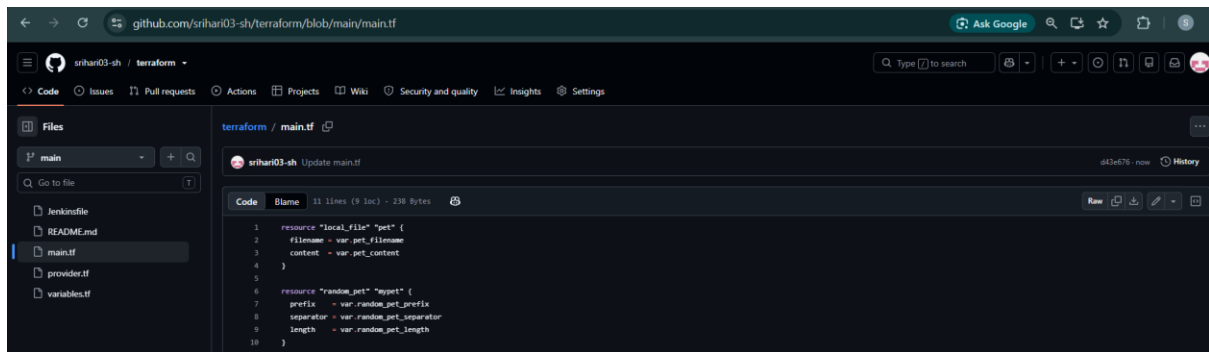
- Open **main.tf** inside repo

Add resources like local_file + random_pet:

```
resource "local_file" "pet" {
  filename = var.pet_filename
  content = var.pet_content
}
```

```
resource "random_pet" "mypet" {
  prefix = var.random_pet_prefix
  separator = var.random_pet_separator
  length = var.random_pet_length
}
```

- Save changes
- Commit and push



- Open **variables.tf** in your Terraform repo
- Define variables with type and defaults:

```
variable "pet_filename" {  
    type = string  
    default = "/tmp/pets.txt"  
}
```

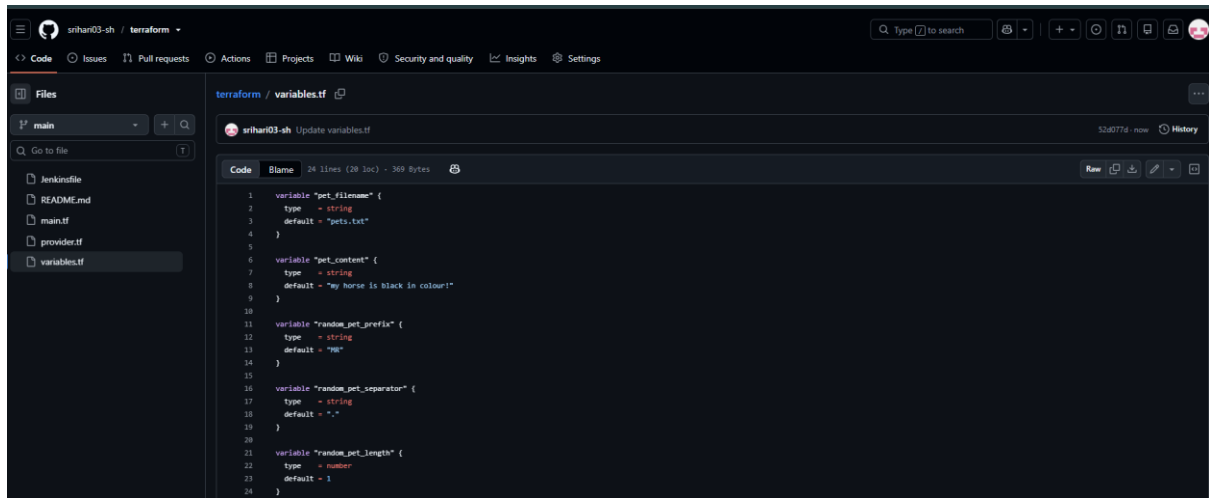
```
variable "pet_content" {  
    type = string  
    default = "my horse is black in colour!"  
}
```

```
variable "random_pet_prefix" {  
    type = string  
    default = "MR"  
}
```

```
variable "random_pet_separator" {  
    type = string  
    default = "."  
}
```

```
variable "random_pet_length" {  
    type = number  
    default = 1  
}
```

- Save and commit changes



- Create a file in repo called **Jenkinsfile**
- Add pipeline stages for Terraform:
 1. Checkout
 2. Terraform Init
 3. Terraform Plan
 4. Terraform Apply
- Example Jenkinsfile flow shown in screenshot uses:
 1. terraform init
 2. terraform plan -out=tfplan
 3. terraform apply --auto-approve tfplan
- Commit Jenkinsfile into GitHub

The screenshot shows the Jenkinsfile editor interface. On the left, a 'Files' sidebar lists 'main', 'Jenkinsfile', 'README.md', 'main.tf', 'providers.tf', and 'variables.tf'. The main editor area is titled 'terraform / Jenkinsfile' and shows a commit from 'mujahed776' with the message 'Rename jenkinsfile to Jenkinsfile'. The code is a Terraform pipeline script:

```
1 pipeline {
2   agent any
3
4   environment {
5     TF_IN_AUTOMATION = "true"
6   }
7
8   stages {
9
10    stage('Checkout from SCM') {
11      steps {
12        git branch: 'main',
13          url: 'https://github.com/mujahed776/terraform.git'
14      }
15    }
16
17    stage('Terraform Init') {
18      steps {
19        sh '''
20          terraform --version
21          terraform init
22          ...
23        '''
24      }
25    }
26
27    stage('Terraform Plan') {
28      steps {
29        sh '''
30          terraform plan -out=tfplan
31          ...
32        '''
33      }
34
35      stage('Terraform Apply') {
36        steps {
37          input message: 'Approve Terraform Apply?', ok: 'Apply'
38          sh '''
39            terraform apply -auto-approve tfplan
40          '''
41        }
42      }
43
44      post {
45        success {
46          echo 'Terraform pipeline executed successfully.'
47        }
48        failure {
49          echo 'Terraform pipeline failed.'
50        }
51      }
52    }
53  }
54 }
```

This screenshot shows the continuation of the Jenkinsfile script from the previous image. The code includes the 'Terraform Apply' stage and the 'post' block for handling success and failure:

```
35      stage('Terraform Apply') {
36        steps {
37          input message: 'Approve Terraform Apply?', ok: 'Apply'
38          sh '''
39            terraform apply -auto-approve tfplan
40          '''
41        }
42      }
43
44      post {
45        success {
46          echo 'Terraform pipeline executed successfully.'
47        }
48        failure {
49          echo 'Terraform pipeline failed.'
50        }
51      }
52    }
53  }
54 }
```

- Create a new Jenkins job: **New Item** → **Pipeline**
- Job name: terraform-task-03
- Go to **Configure** → **Pipeline** section

← ↻ ⚠ Not secure 32.192.178.18:8080/job/terraform-task-03/configure ☆ ⚙️ 👤 ⋮ Chat

 Jenkins

terraform-task-03 ▾ / Configure

🔍 ⚙️

Configure

⚙️ General

🕒 Triggers

📜 Pipeline

🔧 Advanced

Definition

Pipeline script from SCM ▾

SCM ?

Git ▾ ?

Repositories ?

Repository URL ?

https://github.com/srihari03-sh/terraform.git

Credentials ?

- none - ▾ + Add

Advanced ▾

+ Add Repository

← ↻ ⚠ Not secure 32.192.178.18:8080/job/terraform-task-03/configure ☆ ⚙️ 👤 ⋮ Chat

 Jenkins

terraform-task-03 ▾ / Configure

🔍 ⚙️

Configure

⚙️ General

🕒 Triggers

📜 Pipeline

🔧 Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Repository browser ?

(Auto) ▾

Additional Behaviours

+ Add

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

Save

Apply

- Build success

Jenkins

terraform-task-03

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Full Stage View

Stages

Pipeline Syntax

Builds >

Today

#1 3:27 PM

terraform-task-03

Add description

Stage View

Average stage times:

Declarative: Checkout SCM	Declarative: Tool Install	Checkout from SCM	Terraform Init	Terraform Plan	Terraform Apply
408ms	140ms	411ms	1s	698ms	137ms

#1

Sep 07 2057

No Changes

Approve Terraform Apply?

Apply

Abort

137ms

(paused for 30s)

Permalinks

terraform-task-03

Add description

Stage View

Average stage times: (full run time: ~1min 8s)

Declarative: Checkout SCM	Declarative: Tool Install	Checkout from SCM	Terraform Init	Terraform Plan	Terraform Apply	Declarative: Post Actions
408ms	140ms	411ms	1s	698ms	467ms	88ms

#1

Sep 07 2057

No Changes

408ms

140ms

411ms

1s

698ms

467ms

(paused for 1min 16s)

88ms

Permalinks

- Last build (#1), 1 min 16 sec ago
- Last stable build (#1), 1 min 16 sec ago
- Last successful build (#1), 1 min 16 sec ago
- Last completed build (#1), 1 min 16 sec ago

Builds >

Filter

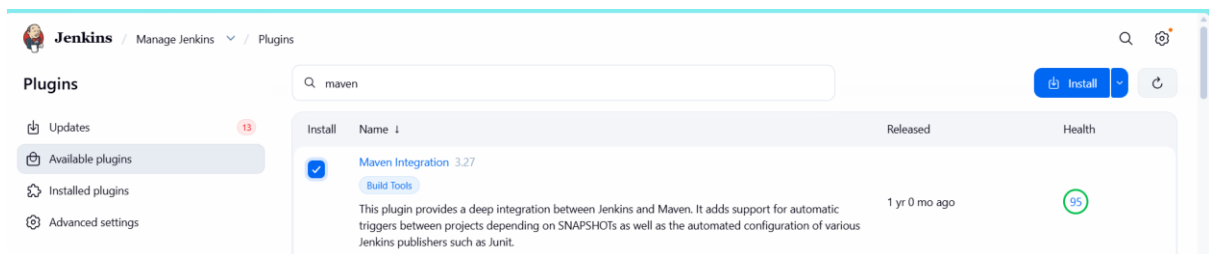
Today

#1 3:27 PM

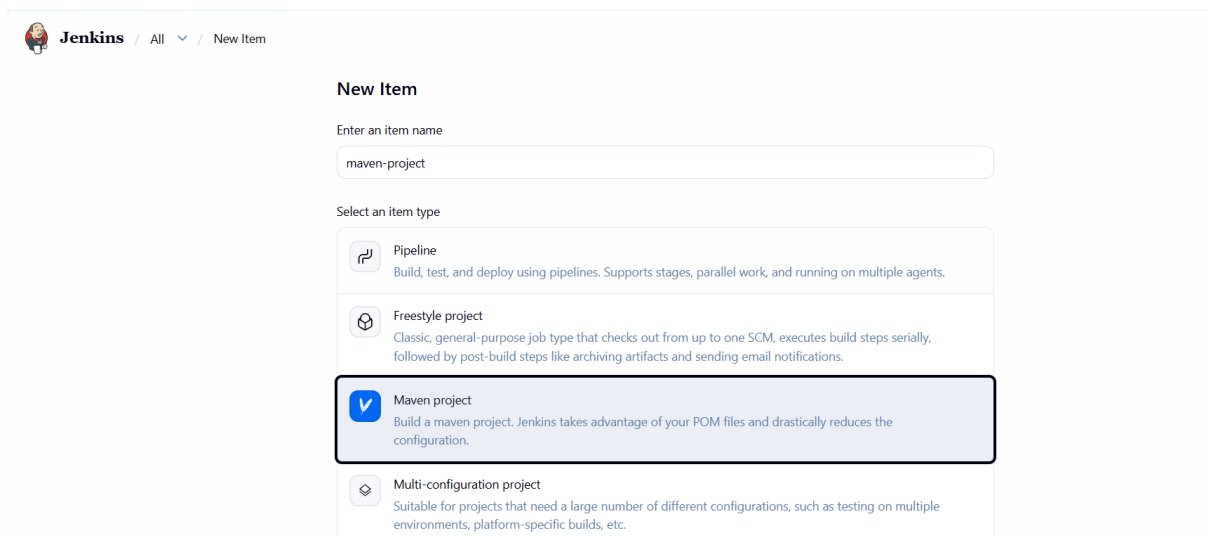
1. Create one Jenkins job using **Maven Project** for the code below with **two stages**:
2. **Stage 1**: Git clone

Stage 2: Maven Compilation **Code**: <https://github.com/betawins/java-Working-app.git>

- Login to **Jenkins Dashboard**
- Go to **Manage Jenkins → Plugins**
- Click **Available plugins**
- Search for **mav**
- Select **Maven Integration**
- Click **Install** (top-right)



- From Jenkins dashboard click **New Item**
- Enter job name: **maven-project**
- Select job type: **Maven project**
- Click **OK**



- Select **Git**
- Paste your GitHub repo URL in **Repository URL**
- If repo is private, add **Credentials**
- In **Branch Specifier**, enter:
- ***/main**

Jenkins / maven-project / Configure

Connect and manage your code repository to automatically pull the latest code for your builds.

Configure

- General
- Source Code Management**
- Triggers
- Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

☐ None

☒ **Git**

Repositories

Repository URL

https://github.com/betawins/java-Working-app.git

Credentials

- none -

+ Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/main

Save Apply

Jenkins / maven-project / Configure

Configure

- General
- Source Code Management
- Triggers**
- Environment
- Pre Steps
- Build
- Post Steps
- Build Settings
- Post-build Actions

☒ Build whenever a SNAPSHOT dependency is built

☐ Schedule build when some upstream has no successful builds

☐ Trigger builds remotely (e.g., from scripts)

☐ Build after other projects are built

☐ Build periodically

☐ GitHub Branches

☐ GitHub Pull Requests

☐ GitHub hook trigger for GITScm polling

☐ Poll SCM

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

☐ Delete workspace before build starts

☐ Use secret text(s) or file(s)

☐ Add timestamps to the Console Output

☐ Inspect build log for published build scans

☐ Terminate a build if it's stuck

☐ Terraform

☐ With Ant

- Go to **Build** section in job configuration
- In **Root POM**, keep:
- pom.xml
- In **Goals and options**, enter Maven goals:
- clean compile
- *(or clean package if you want jar/war output)*

- Click **Save**

Maven installations

Maven installations ^ Edited

+ Add Maven

Maven

Name

Maven

☒ Install automatically ?

Install from Apache

Version

3.9.16

+ Add Installer

Save Apply

Jenkins / maven-project / Configure

Configure

General

Source Code Management

Triggers

Environment

Pre Steps

Build

Post Steps

Build Settings

Post-build Actions

Build

Root POM ?

pom.xml

Goals and options ?

clean compile

Advanced

Post Steps

☐ Run only if build succeeds

☐ Run only if build succeeds or is unstable

☒ Run regardless of build result

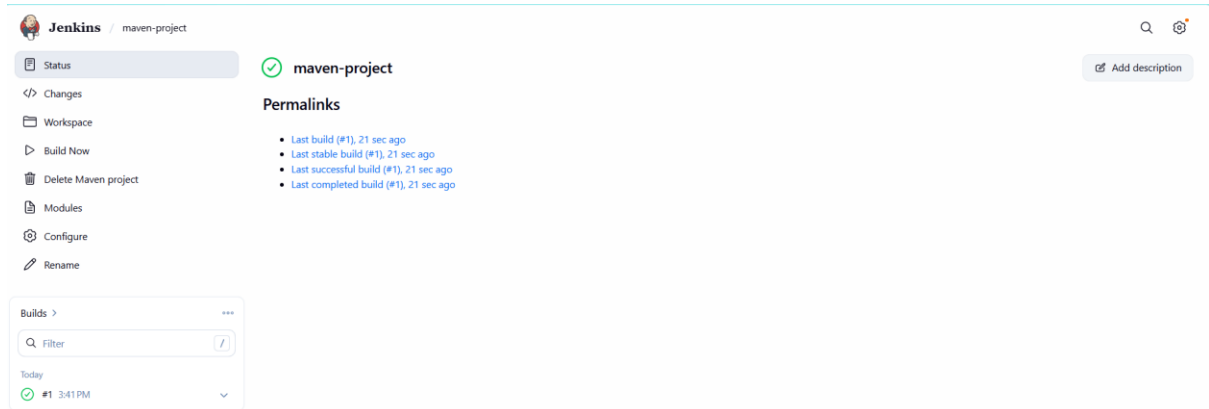
Should the post-build steps run only for successful builds, etc.

+ Add post-build step

Build Continues

Save Apply

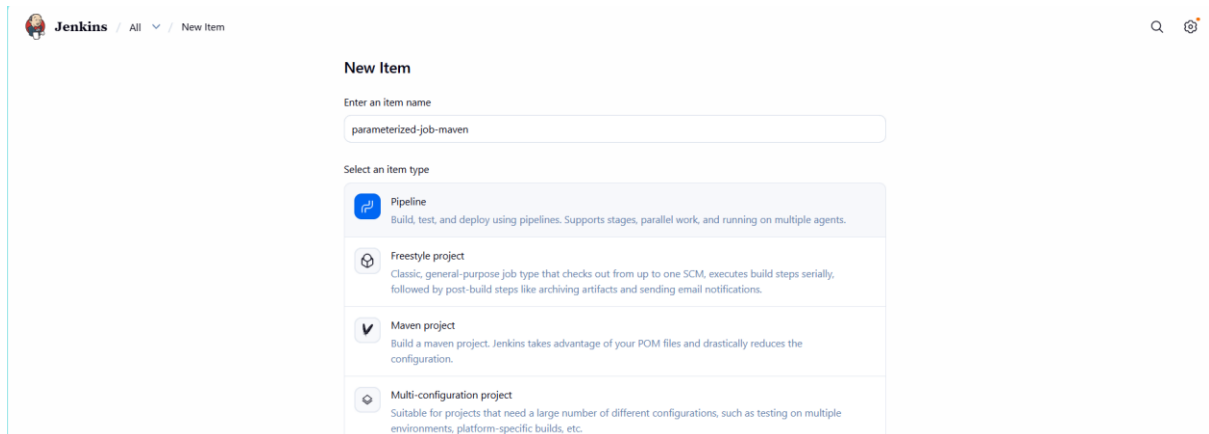
- Click **Build Now**
- Check **Build History** on the left side
- Open build number → Console Output to confirm success



1. Use the same code and create a **parameterized job** in Jenkins with:
2. **Stage 1:** Git clone

Stage 2: Maven Compilation **Code:** <https://github.com/betawins/java-Working-app.git>

- Go to **Jenkins Dashboard**
- Click **New Item**
- Enter Job name: **parameterized-job-maven**
- Select **Pipeline**
- Click **OK**



- Go to **Configure** of the pipeline job
- Tick **This project is parameterized**
- Click **Add Parameter → String Parameter**
- Set Name as:
- GIT_REPO
- Add Default Value (GitHub repo URL), example:
- <https://github.com/betawins/java-Working-app.git>

- Add Description:
Git repository URL to clone the source code from.

Jenkins / parameterized-job-maven / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

☐ Preserve stashes from completed builds ?

☒ This project is parameterized ?

String Parameter

Name ?
GIT_REPO

Default Value ?
https://github.com/betawins/java-Working-app.git

Description ?
Git repository URL to clone the source code from.

Plain text Preview

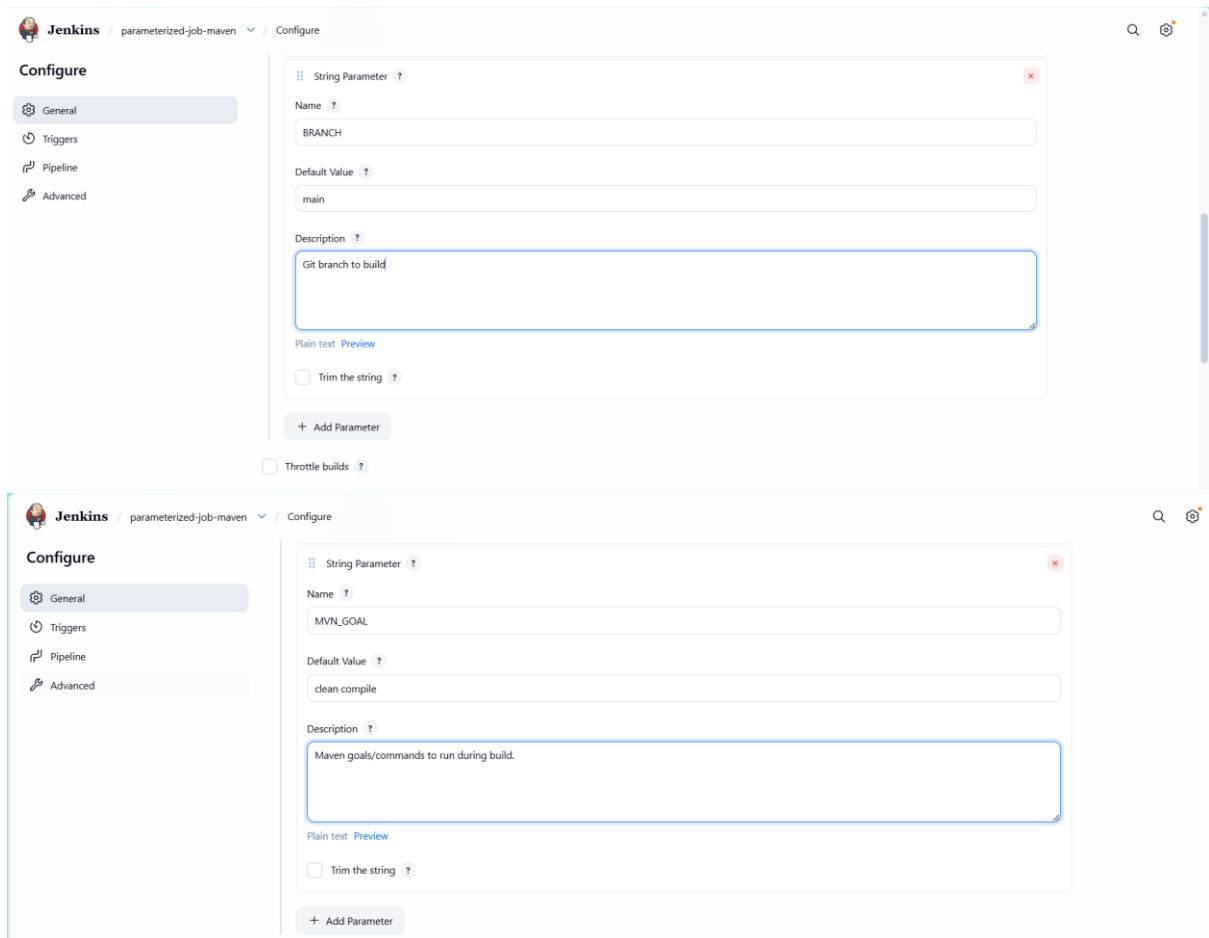
☐ Trim the string ?

+ Add Parameter

☐ Throttle builds ?

Save Apply

- Click **Add Parameter** → **String Parameter**
- Set Name:
- BRANCH
- Default Value:
- main
- Description:
Git branch to build.
- Add another **String Parameter**
- Set Name:
- MVN_GOAL
- Default Value:
- clean compile
- Description:
Maven goals/commands to run during build.



- Scroll to **Pipeline → Definition**
- Choose **Pipeline script**
- Paste the script in the text box (example):

```
pipeline {
```

```
agent any
```

```
tools {
```

```
maven 'Maven'
```

```
}
```

```
parameters {
```

```
string(name: 'GIT_REPO', defaultValue: 'https://github.com/betawins/java-Working-app.git', description: 'Git Repository URL')
```

```
string(name: 'BRANCH', defaultValue: 'main', description: 'Branch Name')
```

```
string(name: 'MVN_GOAL', defaultValue: 'clean compile', description: 'Maven Goals')
```

```
}
```

```
stages {
```

```

stage('Git Clone') {

steps {

git url: "${params.GIT_REPO}", branch: "${params.BRANCH}"

}

}

```

```

stage('Maven Compilation') {

steps {

sh "mvn ${params.MVN_GOAL}"

}

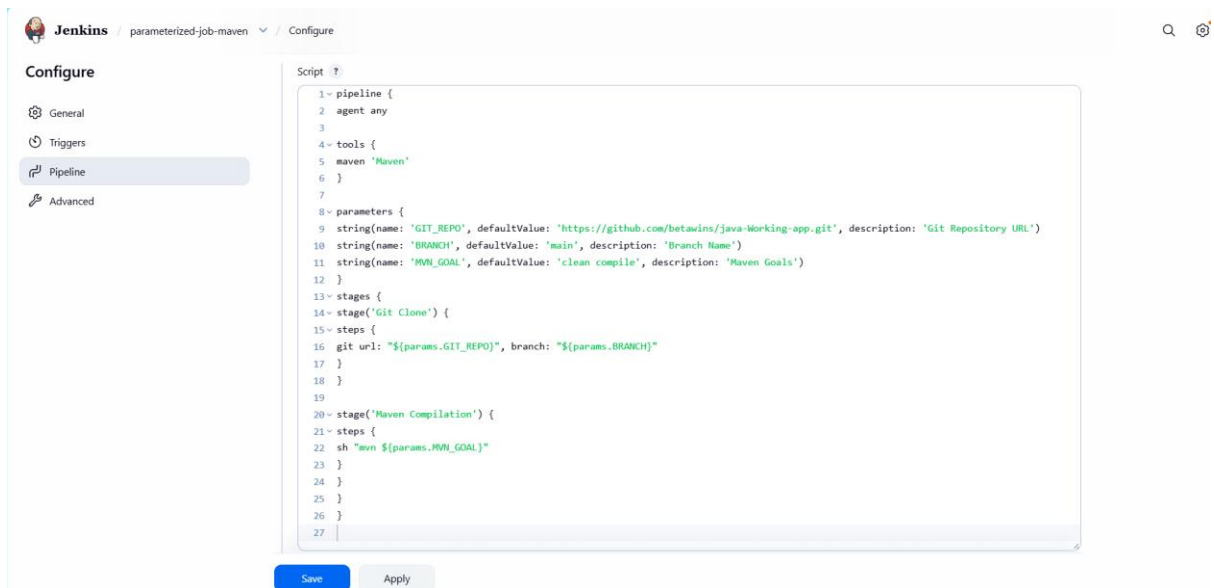
}

}

}

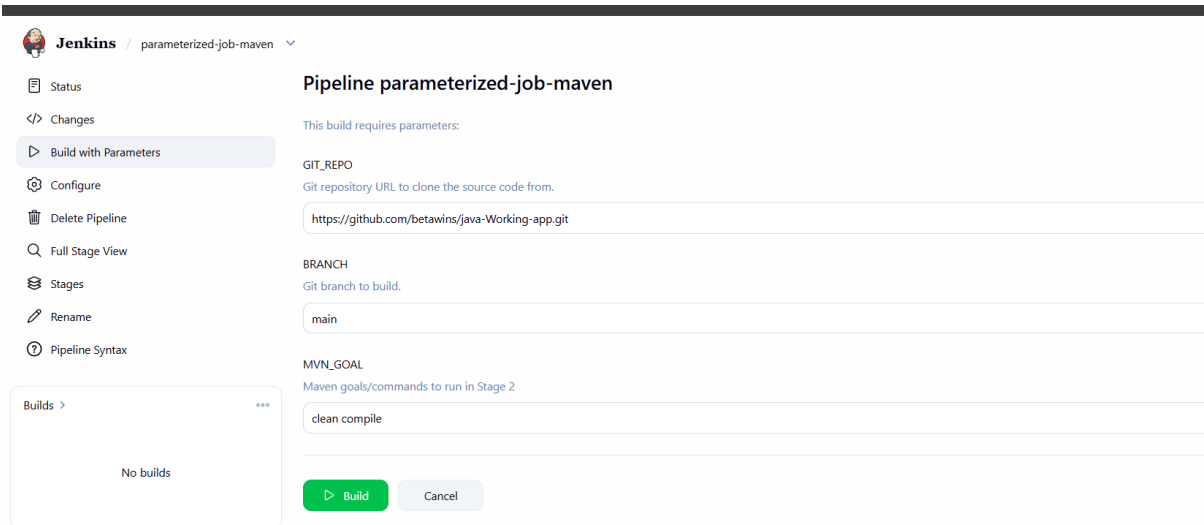
```

- Click **Save**



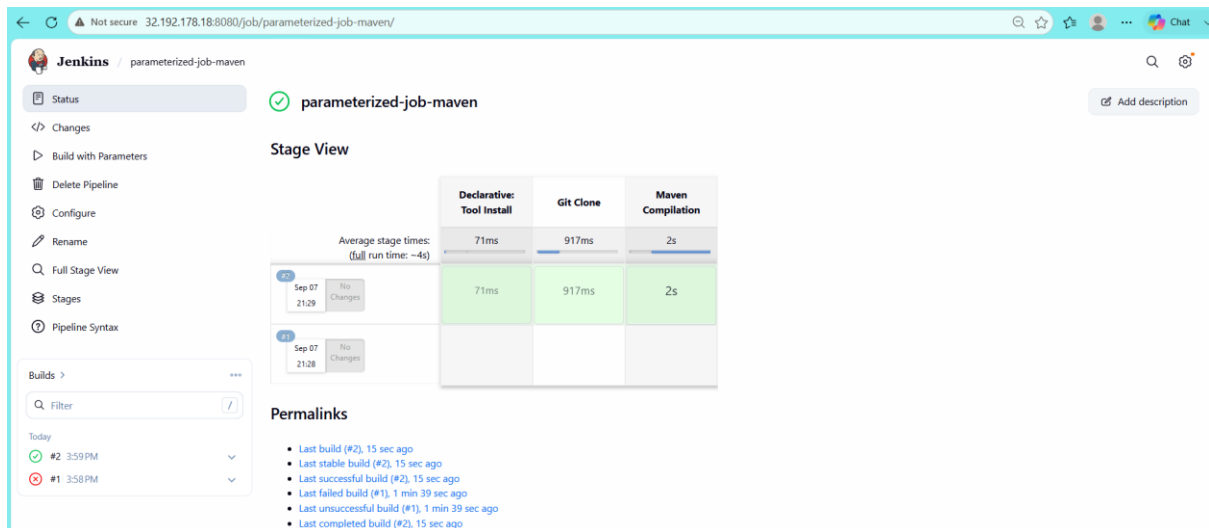
- Open your Jenkins job **parameterized-job-maven**
- Click **Build with Parameters**
- Verify parameters are visible:
 1. GIT_REPO
 2. BRANCH
 3. MVN_GOAL
- Update values if needed

- Click **Build**



The image shows the Jenkins configuration page for a pipeline named "parameterized-job-maven". The left sidebar contains navigation links: Status, Changes, Build with Parameters (selected), Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main area is titled "Pipeline parameterized-job-maven" and includes a note: "This build requires parameters:". Below this, there are three parameter fields: "GIT_REPO" with the value "https://github.com/betawins/java-Working-app.git", "BRANCH" with the value "main", and "MVN_GOAL" with the value "clean compile". At the bottom, there are "Build" and "Cancel" buttons. A "Builds" section on the left shows "No builds".

- After build starts, go to **Stage View**
- Verify stages executed successfully:
Declarative Tool Install
Git Clone
Maven Compilation
- Green boxes indicate the pipeline completed successfully



The image shows the Jenkins "Stage View" for the "parameterized-job-maven" pipeline. The left sidebar is the same as in the previous image. The main area shows a table of stage execution times and a list of builds. The table has columns for "Declarative Tool Install", "Git Clone", and "Maven Compilation". The "Average stage times" are 71ms, 917ms, and 2s respectively. The table shows two builds: #2 (Sep 07 21:29) and #1 (Sep 07 21:28). Build #2 is successful, indicated by a green box. Build #1 is failed, indicated by a red box. Below the table, there is a "Permalinks" section with a list of links to various build details.

	Declarative Tool Install	Git Clone	Maven Compilation
Average stage times: (full run time: ~4s)	71ms	917ms	2s
#2 Sep 07 21:29 No Changes	71ms	917ms	2s
#1 Sep 07 21:28 No Changes			

Permalinks

- Last build (#2), 15 sec ago
- Last stable build (#2), 15 sec ago
- Last successful build (#2), 15 sec ago
- Last failed build (#1), 1 min 39 sec ago
- Last unsuccessful build (#1), 1 min 39 sec ago
- Last completed build (#2), 15 sec ago

1. What are the **global variables** in Jenkins?

In Jenkins, **global variables** are the predefined variables that are available across the Jenkins pipeline to access job, build, and environment related information.

Common Jenkins global variables are:

- **env** → Gives access to environment variables like env.BUILD_NUMBER, env.JOB_NAME.
- **params** → Access parameters passed to the job like params.BRANCH, params.GIT_REPO.
- **currentBuild** → Provides current build details like currentBuild.result, currentBuild.number.
- **scm** → Refers to source code management configuration used in the pipeline.
- **steps / sh / bat** → Pipeline steps used to execute commands.
- **node / workspace** → Helps access agent node and workspace location